AI Resume Analyzer

A Cornerstone Project Report submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

Anurag Nayak (24B11AI014)

Satish Maneddy (24B11AI243)

G Sai Ram (25B21AI047)

Under the Guidance of

Mr. G Siva Sankar, M.Tech, (Ph.D)

Assistant Professor, AIML



DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AND MACHINE LEARNING

ADITYA UNIVERSITY

2024–2028

School of Computing

Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that the project report entitled “AI Resume Analyzer” is submitted by

Anurag Nayak (24B11AI014)

Satish Maneddy (24B11AI243)

G Sai Ram (25B21AI047)

in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial Intelligence and Machine Learning for the academic year 2026–2027.

Project Guide
Mr. G Siva Sankar
M.Tech, (Ph.D)
Assistant Professor, AIML

Head of the Department
Dr. K. Naga Bhargavi
M.Tech, Ph.D.
Associate Professor & HOD, AIML

External Examiner

Signature: _____

DECLARATION

We submit this project report entitled “AI Resume Analyzer” to Aditya University, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech degree in Artificial Intelligence and Machine Learning.

This report describes the implemented local application, its skill extraction and embedding-based matching method, the two-screen frontend, and the verified test results. The demonstration result uses synthetic input and is not a hiring outcome or a model benchmark.

We undertake to verify the implementation details and institutional submission requirements before final submission. We acknowledge the need to comply with the Institutional Academic Integrity and AI Usage Policy, to disclose assistance where required, and to represent experimental evidence accurately.

Place: Surampalem

Date: _____

Submitted By

Anurag Nayak (24B11AI014)

Satish Maneddy (24B11AI243)

G Sai Ram (25B21AI047)

ACKNOWLEDGEMENT

We express our sincere gratitude to our Project Guide, Mr. G Siva Sankar, Assistant Professor, AIML, for the academic guidance and encouragement associated with the Cornerstone Project. We appreciate the opportunity to study a practical application of machine learning and natural language processing through the AI Resume Analyzer project.

We thank Dr. B. Manikyala Rao, Deputy Head of the Department, for academic support and suggestions. We also express our gratitude to Dr. K. Naga Bhargavi, Associate Professor and Head of the Department, AIML, for the departmental learning environment and facilities.

We extend our thanks to Dr. M. V. Rajesh, Associate Dean, School of Computing; Dr. A. Ramesh, Pro Vice-Chancellor, Engineering and Sciences; Dr. S. Rama Sree, Pro Vice-Chancellor, Academics; Dr. G. Suresh, Registrar; Dr. M. B. Srinivas, Vice-Chancellor; and Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor and Management, Aditya University.

We are grateful to the faculty members, laboratory staff, classmates, and our families for their encouragement during our academic work. We also acknowledge the developers and maintainers of the open-source Python tools used for the pilot experiment and the documentation consulted for this report.

Anurag Nayak (24B11AI014)

Satish Maneddy (24B11AI243)

G Sai Ram (25B21AI047)

LIST OF CONTENTS

Chapter	Subject	Page
	Abstract	1
1	Introduction	2
2	System overview	4
3	System implementation	6
4	System architecture	9
5	Data preprocessing	11

Main sections

1.1 Context 2 | 1.3 Problem statement 2 | 1.5 Scope 3

2.1 Functional requirements 4 | 2.4 Constraints 5

3.1 Repository modules 6 | 3.2 API lifecycle 6 | 3.4 Notebook workflow 7

4.1 Components 9 | 4.2 Request contract 9 | 4.4 Failure handling 10

5.1 JD focus 11 | 5.3 PDF extraction 11 | 5.5 Vocabulary 12

LIST OF CONTENTS

Chapter	Subject	Page
6	Methodology and algorithms	14
7	Results and analysis	17
8	Output screens	21
9	Conclusion	26
10	Future scope	27
11	References	28
	Appendix: reproduction steps	29

Main sections continued

6.1 Embedding model 14 | 6.3 Score calibration 14 | 6.5 Assignment 15

6.6 Worked example 15 | 6.7 Projection 16

7.1 Verification 17 | 7.6 Demo result 18 | 7.9 Limits 19

8.1 User flow 21 | 8.2 Upload 22 | 8.3 Map 23

8.4 Matches 24 | 8.5 Missing 25

10.1 Reviewed extraction 27 | 10.2 Calibration 27 | 10.3 Support 27

LIST OF FIGURES AND TABLES

Figures

Figure	Description	Page
4.1	Implemented analysis and display paths	9
8.1	Upload screen with selected demonstration files	22
8.2	Interactive skill map and color legend	23
8.3	Score and matched-skill details	24
8.4	Unmatched job-description skills	25

Tables

Table	Description	Page
2.1	Input and response requirements	4
3.1	Implementation modules	6
3.2	Project team	6
3.3	Runtime inputs and outputs	7
4.1	Analysis stages and data	9
4.2	Browser response contract	10
5.1	Recognition examples and caveats	12
5.2	Extraction behavior	12
6.1	Synthetic scoring example	15
6.2	Credit at the configured floor	16
7.1	Verification evidence	17
7.2	Representative regression checks	19

ABSTRACT

AI Resume Analyzer is implemented as Matchspace, a local web application that compares named technical skills in a job description with those in a resume. A user uploads a UTF-8 text job description and a selectable-text PDF resume, then receives a score, matched and unmatched skills, and an interactive three-dimensional view. The application is intended to make the comparison inspectable; it does not rank applicants or predict hiring outcomes.

The backend extracts skills with a curated alias vocabulary, embeds each distinct label using the pretrained all-MiniLM-L6-v2 sentence-transformer model, computes full-vector cosine similarity, and converts similarity above a 0.60 floor into fractional points. A one-to-one assignment selects the set of pairs with the greatest total credit. Principal component analysis projects the same embeddings into three dimensions only for display; the projected coordinates never determine the score.

The report documents the implemented upload API, in-memory processing, PDF layout extraction, score calculation, browser interface, and verification. In a synthetic demonstration with seven job-description skills, five exact pairs earned five points and the interface displayed 71.43%. Automated checks passed: 31 Python tests and six JavaScript tests. These checks establish functional behavior for the tested cases, not validity against recruiter judgments or a representative resume dataset.

Keywords: resume analysis, job description, skill extraction, sentence embeddings, cosine similarity, one-to-one matching, PCA visualization.

CHAPTER 1
INTRODUCTION

1.1 Context

A resume and a job description describe skills in different styles. One may say “K8s” where the other says “Kubernetes”; one may name Docker while the other asks for containerization. Direct text search can miss related expressions, while a single opaque percentage can conceal what was actually compared. The project makes those comparisons visible at the skill level.

Matchspace is a local academic prototype. Its input is not a recruiter database, and its output is not an employment decision. The application extracts named technical terms from a supplied job description and resume, embeds those terms in a shared vector space, and presents the selected pairs with their credits. Each stage has a distinct role: extraction determines what enters the score, embeddings measure relatedness, and the browser presents the result.

1.2 Why this approach

A curated vocabulary offers predictable labels and aliases. A pretrained embedding model adds graded similarity between recognized skills without requiring this project to train on personal resumes. One-to-one pairing prevents a single resume term from satisfying several job-description requirements. The final display exposes matches and missing items so a reader can challenge the score rather than accepting it uncritically.

1.3 Problem statement

The core problem is to compare two uploaded documents consistently enough for a user to inspect where their technical terms overlap. The input may contain formatting noise, repeated words, unrelated article content, or short aliases. The result must therefore show which labels were detected, how pairs were credited, and which job-description labels remained without a qualifying pair.

A percentage alone would be misleading if the denominator came from an incomplete skill list. The initial SRE example exposed this issue: only three skills were recognized before the vocabulary and PDF layout extraction were improved. The system now recognizes additional SRE terms, but it still relies on a finite vocabulary. That limitation is documented and remains visible through the matched and unmatched lists.

1.4 Project objectives

- Accept a text job description and a selectable-text PDF resume through a two-screen web interface.
- Identify distinct technical skill labels and preserve readable display names.

- Compute pair credit from the original embedding vectors and assign each resume skill at most once.
- Show the result as a score, itemized matches and missing skills, and an interactive three-dimensional projection.
- Reject unsupported or unreadable uploads with clear feedback and keep upload content in memory during an analysis.

1.5 Scope and exclusions

The current application handles English technical skill labels covered by the curated alias map. It accepts a UTF-8 .txt job description of at most 250 KB and a selectable-text .pdf resume of at most 5 MB. The server reads the PDF with pypdf in layout mode, which can preserve multiword phrases in some layouts better than plain extraction. Scanned image-only PDFs have no selectable text and are rejected with an OCR explanation.

The application does not parse DOCX resumes, train a classifier, infer job categories, verify experience, assess proficiency, or use protected personal characteristics in scoring. It does not save uploaded files or maintain a persistent vector database for the web flow. The original notebooks can save extracted vectors and scores as local JSON artifacts, but that is a separate batch workflow.

The word “match” means the configured embedding similarity passed the scoring floor after extraction. It does not mean that two dots literally collide in three-dimensional space, that a skill was demonstrated in work experience, or that an employer would accept it as equivalent. The user must inspect the terms in context.

CHAPTER 2
SYSTEM OVERVIEW

2.1 Functional requirements

The browser must provide one upload screen and one result screen. The first screen identifies the required formats and sizes, supports browsing or dropping files, and enables analysis only when both files are selected. The result screen shows the numeric score, the number of scored pairs, the total extracted job-description skills, and tabs for matched and unmatched items. A user can begin another analysis without reloading the server.

Area	Required behavior
Input	Accept JD .txt and resume .pdf; reject missing, empty, wrong-type, or oversized files.
Analysis	Extract distinct skills, embed labels, score one-to-one pairs, and project points for display.
Result	Return score, points, total, matched pairs, unmatched labels, and 3D coordinates.
Navigation	Allow a new analysis and preserve a useful browser Back action during the session.

Table 2.1 Input and response requirements

2.2 Input validation and error states

The frontend checks file extensions, empty files, and size limits before sending a request. The backend repeats these checks because client-side validation cannot be trusted as the only gate. The API accepts multipart form data with fields named jd and resume. A malformed request, invalid UTF-8 job text, corrupt PDF, image-only PDF, or job description with no supported technical skills receives an error response instead of a fabricated score.

If no recognized resume skill remains after extraction, the system returns zero points and shows the job-description skills as unmatched. This is different from a PDF extraction failure: the first case has readable text with no recognized labels; the second has no usable text. Keeping these cases separate helps the user decide whether to revise the document or inspect the vocabulary.

2.3 Visible feedback

The interface announces the current processing phase, displays errors near the form, and keeps the result details available in text below the visualization. The score is shown to two decimal places when needed; its progress animation must settle on the backend value. Color conveys the three visual roles, while labels and tabular skill rows convey the same result without relying only on color.

2.4 Quality and operating constraints

The application is designed to run locally on 127.0.0.1. Its static frontend uses HTML, CSS, and JavaScript without a build step; the Python server serves those files and the analysis endpoint. The embedding model is loaded only after an upload passes input validation and is cached for later analyses in the running process. The first successful analysis can therefore take longer than later ones, especially if the pretrained model must be downloaded into the local cache.

The development checks cover score calculations, matching assignment, projection behavior, API validation, and selected browser scene functions. The current report does not claim throughput, browser compatibility across all devices, accuracy on a labeled resume dataset, or calibrated hiring relevance; those require a separate evaluation plan.

Constraint	Current treatment
Privacy	Upload bytes are processed in memory and are not saved by the web app.
Model dependency	A pretrained model is loaded from local cache or fetched on first use.
Accessibility	Text detail rows, keyboard camera controls, and reduced-motion handling are implemented.
Deployment	Local development server is verified; public deployment is outside this report.

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 Repository modules

The implementation is intentionally small. matcher.py owns the extraction and mathematical core; server.py owns the HTTP boundary; frontend/ owns the static browser interface; tests/ contains Python behavior checks; and the frontend test files check scene math and score formatting. Two notebooks retain a local batch route for storing extracted vectors and scores as JSON.

File or directory	Role
matcher.py	JD focus, PDF text, aliases, embeddings, cosine score, assignment, PCA projection
server.py	Static files, upload validation, analysis endpoint, model lifecycle
frontend/	Upload screen, score screen, canvas scene, styles, browser behavior
tests/	Python extraction, scoring, projection, and server checks
01 and 02 notebooks	Optional local extract/store and score workflow

Table 3.1 Implementation modules

3.1.1 Project team

Name	Roll number
Anurag Nayak	24B11AI014
Satish Maneddy	24B11AI243
G Sai Ram	25B21AI047

Table 3.2 Project team

3.2 API lifecycle and model loading

Running server.py starts a local ThreadingHTTPServer. A GET to /api/health confirms that the process is alive. The same server serves the frontend files and handles POST /api/analyze. The upload parser reads the two multipart fields, then analyze_upload performs validation, extraction, embedding, scoring, and projection. The JSON response contains only the fields needed by the browser display.

The embedding model is not loaded at server start. It is requested after a valid upload has yielded JD and resume skill lists. The model loader prefers a local cache and otherwise downloads the pretrained model. A lock protects the first load. This design avoids repeated model startup cost within a process, but it does not measure memory use or parallel throughput under production traffic.

3.2.1 Input records and model state

The application does not train on a resume dataset. Its input records are the two user-supplied files for one request, and the pretrained encoder is read from the local model cache or downloaded when needed. The job text and PDF bytes are discarded after the request completes. Synthetic documents are used only as repeatable fixtures for tests and screenshots. This distinction prevents a small browser demonstration from being presented as model training evidence.

For every accepted request, the server keeps the extracted skill objects in memory. Each object carries its source, visible label, canonical label, and model vector. The response returns labels, pair credits, and three-dimensional coordinates but not the original PDF body or 384-dimensional vectors. The notebook route is different: it explicitly serializes the extracted items to local JSON for teaching and repeatability.

Input	Representation	Handling
JD .txt	UTF-8 text, at most 250 KB	Decode; select the relevant skills section; recognize labels.
Resume .pdf	Selectable text, at most 5 MB	Read uploaded bytes; use layout extraction; recognize labels.
Model	Pretrained all-MiniLM-L6-v2	Load lazily once per server process.
Result	JSON response	Return score, positive pairs, missing skills, and 3D points.

Table 3.3 Runtime inputs and outputs

3.3 Dependencies

The Python requirements include sentence-transformers, pypdf, NumPy, SciPy, and JupyterLab. JupyterLab supports the notebook route; the web frontend itself needs no Node build. The implementation uses Python’s standard-library HTTP server for local demonstration rather than a production web framework. Model and dependency versions should be pinned for any later deployment or externally reproducible benchmark.

3.4 Notebook workflow and saved vectors

The notebooks provide a separate reproducible exercise. The first reads input/JD.txt and input/resume.pdf, extracts and embeds recognized items, then saves output/extracted.json. The second loads that file, computes the score, and saves output/score.json. These JSON files are local artifacts of the notebook workflow; they are not a vector database and are not used to remember web-app uploads.

The distinction matters for calibration. The score depends on the model, alias vocabulary, expansion strings, and floor. If any of these change, previously saved vectors or scores may no longer represent the current method. The notebooks should be rerun from the extraction stage,

and the configuration should be recorded alongside reported results. The browser flow calculates fresh vectors for each accepted upload.

3.5 Reproduction

From the repository root, create a Python environment, install requirements.txt, and run server.py. Open http://127.0.0.1:8000 in a browser. The first valid request may need a model download and takes longer. To check the implementation, run the Python test discovery command and the JavaScript test command listed in the appendix. The app handles uploads in memory; a result is not restored after a fresh browser load.

3.6 Local run sequence

The user selects the two supported files and submits the form. The browser sends multipart data to the local API, which validates the request before loading the encoder. When the response arrives, the browser creates the score card and scene from the same result object. Selecting New analysis resets file selection and result state. A direct refresh of the results route returns to upload because the server does not save per-user history.

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 Components and responsibilities

The application has three code layers. The frontend gathers files and renders the interactive view. server.py validates multipart uploads, serves the static assets, and coordinates the analysis. matcher.py performs text extraction, skill recognition, embedding, scoring, and projection. The browser receives only the result object needed for the score and scene; the raw resume text is not included in that response.

Stage	Input	Output
Upload and validation	JD text file and resume PDF	Validated bytes or an error
Text extraction	Validated bytes	JD text and PDF text
Skill recognition	Readable text	Distinct JD and resume skill labels
Embedding and scoring	Skill labels	Vectors, credits, and assignment
Projection and display	Both vector sets and scored pairs	3D coordinates and browser view

Table 4.1 Analysis stages and data

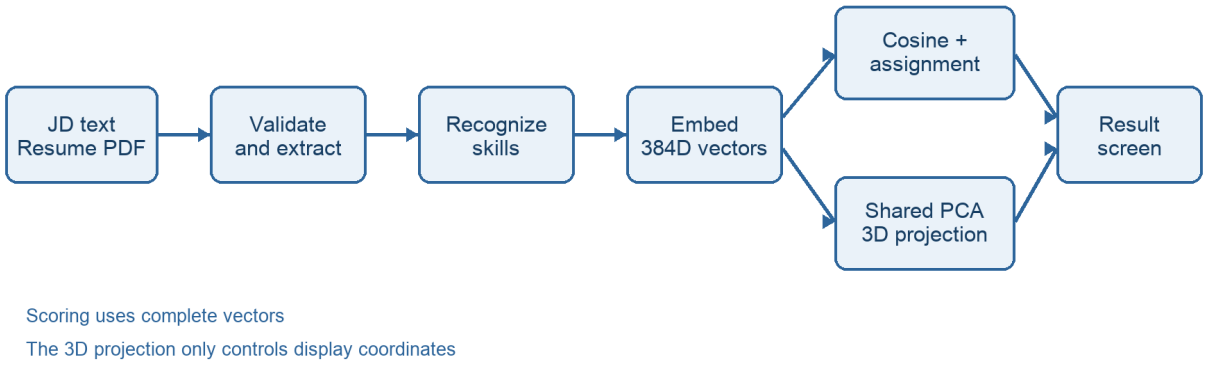


Figure 4.1 Implemented analysis and display paths

4.2 Request and response contract

The browser submits a multipart POST to /api/analyze. The two fields are jd and resume. A successful JSON response includes score, points, total, matched, unmatched, points_3d, and resume_skill_count. Each matched row contains the visible JD and resume labels, their indices, cosine similarity, and awarded credit. The indices connect the scored pair to its two projected points without repeating the high-dimensional vectors in the browser response.

The health endpoint /api/health returns a small status object. The server also serves / and /results with the same HTML shell. Client-side navigation changes the route after a successful analysis. Because the result is held in browser memory, a fresh direct load of /results returns to the upload screen rather than reconstructing a previous private analysis.

4.2.1 Response fields

The browser needs a compact, inspectable contract. It uses the score and point total for the score card; matched and unmatched for the detail tabs; and points_3d for the canvas. Pair indices refer to the order of the two point arrays. The scene adds purple markers only for returned positive-credit pairs. A change to the backend response shape therefore requires a corresponding frontend test and UI update.

Field	Type	Meaning
score	number	Two-decimal percentage from unrounded credits.
points / total	number / integer	Earned credit and JD-skill denominator.
matched	array	Assigned positive-credit JD and resume pairs.
unmatched	array	JD labels without a selected positive pair.
points_3d	object	JD and resume coordinates for display only.

Table 4.2 Browser response contract

4.3 Contract example

{"score": 71.43, "points": 5.0, "total": 7, "matched": [...], "unmatched": ["Terraform", "Observability"], "points_3d": {"jd": [...], "resume": [...]}}

The example is a shortened representation of the synthetic run documented in Chapter 7. It illustrates field meanings; it is not a complete stored response.

4.4 Processing sequence and failure handling

The server first limits the request body, parses the multipart envelope, checks both filenames and sizes, decodes the JD as UTF-8, and extracts selectable PDF text. Only then does it identify skills and ask for the embedding model. This order avoids loading a large model for a malformed request. The model object is reused in the server process under a lock so concurrent requests do not create separate initial loads.

A valid job description with no recognized technical skill cannot establish a denominator and receives a clear validation message. A valid resume without recognized skills is still analyzable and yields a zero score with missing JD labels. Unexpected server errors are logged and returned to the browser as a generic failure message; document contents are not deliberately written to a database or report file by the web path.

This flow separates input errors from internal failures. It also makes the principal limitation explicit: quality depends on the text obtained from the PDF and on the curated recognition list. Neither the embedding model nor the visual display can recover skills that were never extracted.

CHAPTER 5
DATA PREPROCESSING

5.1 Job-description focus

The job-description reader accepts UTF-8 text. Before recognizing skills, focus_jd_text selects an explicit Required Qualifications or Requirements section when one is present. If neither exists, it can select a named skills section. This reduces accidental counting of tool names in benefits, site navigation, examples, and article footers. With no recognized section heading, the implementation falls back to the available text, so the extracted list should be reviewed.

This rule was important in the SRE example that motivated the revision. The supplied text was a general explanatory article rather than a single employer posting. Focusing its named skills section produced a more meaningful denominator than scanning salary details and related links. A score from such an article remains a demonstration of textual overlap, not an assessment against a real vacancy.

5.2 JD count

Each canonical skill appears once even if the text repeats a label or uses several aliases. The score denominator is the number of distinct recognized JD skills after this filtering. Preferred qualifications are excluded when a required-qualification section is identified. The frontend lists the resulting count so the user can see whether that denominator seems plausible.

5.3 Resume PDF extraction

The web server reads a PDF directly from the uploaded bytes with pypdf [5]. The extractor asks for layout mode and normalizes repeated whitespace within each line. This can keep phrases such as “incident management” together when the default extractor splits words onto separate lines. The path-based notebook reader uses the same text extraction helper, keeping both workflows consistent.

The system rejects a PDF that yields no selectable text. That error is reported as an OCR need rather than as a zero skill match. Layout extraction is still not a full document-understanding system: columns, graphics, unusual fonts, and tables can reorder or omit text. The report therefore treats the extracted labels as inspectable output rather than verified claims about a person.

5.4 Resume contact filtering

The skill recognizer skips resume lines that appear to be contact details or links. This reduces the chance that a name, social profile, or URL contributes accidental skill words. This filtering is conservative and line based; it does not anonymize a resume, remove all personal information,

or decide whether an experience statement proves competence. The browser result uses skill labels, not the full resume body.

5.5 Curated vocabulary and aliases

SKILL_ALIASES in matcher.py is the predefined vocabulary. Entries map visible variants to a canonical skill, for example React.js to React and K8s to Kubernetes. Recognition uses phrase boundaries and prefers longer aliases when matches overlap. The list covers general software tools and expanded SRE terms such as Bash, automation, troubleshooting, monitoring, logging, observability, incident management, root cause analysis, postmortem, on-call, SLO, Terraform, Prometheus, Grafana, Splunk, and GitOps.

Input phrase	Canonical label	Interpretation
K8s	kubernetes	Known abbreviation
shell scripting	bash	Curated alias; review context
containerization	containerization	Distinct from Docker
Docker	docker	May earn partial semantic credit against containerization
React.js	react	Alias used for deduplication

Table 5.1 Recognition examples and caveats

5.5.1 Boundary and alias examples

The recognizer searches each line with case-insensitive phrase boundaries. Longer aliases are tested first, so a phrase such as “restful api” does not also count a shorter overlapping alias. Canonical deduplication then collapses repeated mentions. The visible label is retained from the earliest accepted mention, which is why the result may show “K8s” even though the canonical key is Kubernetes. Contact and URL-like resume lines are skipped before this pass.

Text example	Expected treatment	Reason
K8s and Kubernetes	One Kubernetes skill	Alias deduplication
React.js in a project	One React skill	Known spelling variant
Required qualifications: Docker Preferred qualifications: AWS	Docker only	Required-section focus
Image-only resume PDF	Extraction error, not zero score	No selectable text

Table 5.2 Extraction behavior on representative inputs

The rule is deliberately narrower than open-vocabulary skill extraction. It reduces some false matches, but it also misses unknown names and context-dependent equivalents. A later editable extraction review would let the user correct both types of error before the denominator is fixed.

5.6 Recognition limits and review

The vocabulary is finite. A new tool, spelling variant, or domain term that is not listed can be absent from the result even when it appears in both files. Conversely, a short term may be recognized in a context where it is only mentioned, not demonstrated. The implementation does not use a language model to infer every possible skill from arbitrary prose; it extracts supported labels and compares those labels semantically.

This distinction matters when interpreting a high score. If only a few requirements were recognized, matching all of them can produce 100% despite many unrecognized requirements. A reliable review should inspect the JD skill count and the matched and missing lists, then add missing vocabulary with tests. Future evaluations should measure extraction precision and recall against independently annotated resumes and job descriptions.

In the current prototype, the user cannot edit the extracted list in the browser before scoring. That is a limitation, especially for copied articles and ambiguous requirements. An editable preview is a sensible next step, but it is not represented as implemented in this report.

CHAPTER 6
METHODOLOGY AND ALGORITHMS

6.1 Embedding model

The matcher uses the pretrained sentence-transformers/all-MiniLM-L6-v2 model. Its model card describes a 384-dimensional dense embedding for short text [1]. Each distinct JD and resume skill label is encoded in the same vector space with normalized embeddings. The project does not fine-tune this model and does not train on uploaded resumes.

A small expansion map replaces opaque labels before embedding. For example, K8s is encoded as “kubernetes,” CI/CD as “continuous integration and continuous deployment,” and Docker as “Docker containerization.” The visible labels remain those found in the uploaded text. Expansions make short abbreviations more informative to the encoder; they are explicit rules and can affect similarity, so changes require regression checks.

6.2 Vector storage

The live web path keeps vectors in memory long enough to score and project one request. It does not insert them into a vector database. The notebook path can save vectors in output/extracted.json for a repeatable local exercise. After changing the vocabulary or expansion map, notebook users must rerun extraction to regenerate that file; old stored vectors do not automatically recalibrate.

6.3 Similarity and point calibration

For each JD vector j and resume vector r, cosine similarity is $(j \cdot r) / (\|j\| \times \|r\|)$. A value near one indicates a similar direction in the model space. The scoring function clamps the calculated value into the expected numeric range and uses zero credit for pairs at or below the configured floor of 0.60. Above that floor, credit increases linearly to one at cosine one: credit = clamp((cosine − 0.60) / 0.40, 0, 1).

The overall percentage is 100 × (sum of awarded pair credits) / (number of distinct JD skills). Credits are calculated from unrounded cosine values; displayed similarities and pair credits are rounded only for the response. Thus, a visually rounded 1.00 credit need not imply identical language unless its unrounded similarity is one. The floor is a project calibration choice, not a universally validated threshold.

6.4 Interpreting the number

A score of 70% means the accepted pair credits total about 70% of the recognized JD-skill denominator under this model and floor. It is not a probability of being hired, a measure of job

performance, or a direct percentage of resume sentences that match. Model changes, alias expansions, or a different JD section can change the result even for unchanged files.

6.5 One-to-one skill assignment

The system constructs a matrix of possible pair credits between JD skills and resume skills. It uses SciPy linear_sum_assignment to choose the set of pairs that maximizes total credit while assigning each resume skill at most once [3]. Additional zero-credit dummy columns allow a JD skill to remain unmatched. This prevents one broad resume label from covering several JD requirements simply because it resembles each of them.

A greedy approach that picks the strongest available pair for each JD line can lose total score when a second JD skill has only one good candidate. Global assignment considers the whole matrix. The result records the two indices, labels, cosine similarity, and credit for each selected positive-credit pair. JD skills without a positive selected pair enter the unmatched list.

Principle	Effect on report output
One resume skill used once	No repeated credit from the same detected resume term.
Dummy unmatched options	A weak pair does not have to be forced into the result.
Positive-credit pairs only	The Matches tab corresponds to contributions to the score.
Global maximization	The chosen set has the largest total credit under the matrix.

6.6 Worked scoring example

For a reproducible browser demonstration, a synthetic JD listed Python, Bash, Docker, Kubernetes, Terraform, CI/CD, and Observability. A synthetic resume listed Python, Bash, Docker, Kubernetes, CI/CD, Prometheus, and Linux. The application recognized seven JD skills. Five exact label pairs received one point each. Terraform and Observability had no qualifying assigned pair in that run.

JD skill	Selected resume skill	Credit
Python	Python	1.00
Bash	Bash	1.00
Docker	Docker	1.00
Kubernetes	Kubernetes	1.00
CI/CD	CI/CD	1.00
Terraform	None	0.00
Observability	None	0.00

Table 6.1 Synthetic scoring example captured from the running application

Total credit = 5.00; denominator = 7; displayed score = 100 × 5/7 = 71.43%. This is one test pair used to document the interface. It does not estimate matching accuracy over a real resume population.

6.6.1 Sensitivity of the scoring floor

The calibration formula makes the threshold visible. With a floor of 0.60, a cosine of 0.60 earns zero; 0.70 earns 0.25; 0.80 earns 0.50; 0.90 earns 0.75; and 1.00 earns one point. These are arithmetic examples, not measured skill-pair similarities. They show why calling cosine 0.80 an “80% match” would misdescribe this implementation.

Cosine	Pair credit	One-skill score
0.60	0.00	0%
0.70	0.25	25%
0.80	0.50	50%
0.90	0.75	75%
1.00	1.00	100%

Table 6.2 Credit produced by the configured floor

Because credits are combined through one-to-one assignment, the final score is not the average of all pairwise similarities. The denominator is the distinct recognized JD-skill count. A different vocabulary can change that denominator before any vector comparison occurs. This is why the interface must show the count and unmatched list beside the headline percentage.

6.7 Three-dimensional projection

After scoring, the server combines the JD and resume vectors and centers them together. A singular value decomposition finds principal axes, and the vectors are projected onto the first three axes. If fewer than three axes exist, the coordinates are padded. The projected points are scaled for display. NumPy provides the SVD operation used by the implementation [4].

The two sets must share one projection; projecting them separately would make their plotted positions incomparable. Even with a shared projection, three dimensions discard information from the original 384-dimensional model space. Two points that appear close on screen may be less similar in the complete space, and vice versa. Therefore the score uses full vectors, while the 3D scene is an explanatory view.

6.8 Purple markers

A purple marker and connecting line are drawn for each selected positive-credit pair. They represent a scored relationship, not a literal coordinate collision. The browser calculates a midpoint between the pair’s projected positions for its marker. Blue denotes JD skills, red denotes resume skills, and the labels and match rows remain the source of precise pair information.

CHAPTER 7
RESULTS AND ANALYSIS

7.1 Verification approach

The verification record separates automated unit behavior from the synthetic browser walkthrough. Unit tests check code paths with controlled input and fake or fixed vectors; they do not measure semantic accuracy on real hiring data. The browser walkthrough checks that the actual two-screen flow displays the score and skill rows for one constructed pair of files.

Evidence	Observed result
Python unit suite	31 tests passed, including PDF extraction, aliases, scoring, projection, and API errors.
JavaScript unit suite	6 tests passed for camera math, collision markers, hit testing, and score formatting.
Synthetic browser flow	Upload completed; results showed 71.43%, five matches, seven JD skills, two missing.
Frontend screenshots	Figures 8.1–8.4 captured from the running local app.

Table 7.1 Verification evidence recorded for this revision

7.2 Backend test coverage

The Python tests exercise skill alias deduplication, required-section focus, exclusion of article footer text, SRE term recognition, and layout-mode extraction of multiword PDF phrases. Error cases include corrupt and image-only PDFs, empty job text, incorrect file types, oversized or missing multipart fields, and a resume with no recognized skills. A regression test checks that a callable SentenceTransformer object is not mistaken for a model-loader function.

Scoring tests use known vectors to verify the similarity floor, fractional credit, unrounded score calculation, one-to-one assignment, duplicate-item handling, and the relationship between Docker and containerization. Projection tests check a shared space and the case of identical vectors. These tests establish mathematical and code-path behavior; they do not establish that all labels in a real document are extracted correctly.

7.3 Test boundaries

A passed test means its specified input produced its expected output in the current environment. It should not be generalized to all PDF layouts, every technical synonym, or a particular resume population. A larger evaluation needs labeled job requirements and resumes, independent reviewers, and error analysis by skill category.

7.4 Frontend checks

The JavaScript tests verify that zero camera rotation preserves coordinates, perspective keeps the center stable, scored pairs create purple markers, hit testing resolves overlapping points, keyboard controls respect camera bounds, and score formatting does not round a value upward to a false perfect score. The browser demonstration then verifies a real upload and navigation path in addition to these isolated functions.

The screen uses a canvas for the 3D representation but repeats the information as text rows. A user can inspect matched and missing skills without hovering over points. The app also provides descriptive labels for the map and controls and honors reduced-motion preference for automatic rotation. These are useful accessibility measures, although a formal accessibility audit with assistive technologies has not been completed.

7.5 Demonstration input

The synthetic input deliberately includes five shared terms and two JD terms without qualifying pairs. This checks both the Matches and Missing views and makes the 5/7 arithmetic easy to audit. The screenshots in Chapter 8 were captured from this local run. They must not be interpreted as a validated ranking of a real person.

7.6 Demonstration result

The running app returned five positive-credit pairs: Python, Bash, Docker, Kubernetes, and CI/CD each matched the same visible label in the synthetic resume and received 1.00 point. Terraform and Observability appeared in the Missing tab. The total was 5.00 of seven possible skill points, so the score displayed as 71.43%. The screenshot also shows the 3D projection and color coding for the same response.

This result confirms that the backend score, browser count, pair list, and missing list agree for this constructed example. The project also handled a separate SRE article and resume during development after expanding its vocabulary and improving PDF layout extraction. Because that article is not a vacancy, the present report uses a synthetic, clearly specified pair for reproducible explanation rather than presenting the article score as job-fit evidence.

7.7 Verification cases

The backend tests cover both successful paths and failures. The following cases illustrate what was checked without claiming a statistical evaluation over real resumes. Each check has a narrow expected outcome that can be repeated after a code change. Browser screenshots verify the rendered path for one synthetic file pair; they do not replace the unit checks for malformed files or boundary behavior.

Case	Expected behavior	Observed
Recognized alias	Canonical skill appears once	Pass
One resume skill vs two JD skills	At most one assignment	Pass
Wrong extension or missing file	Input error	Pass
Image-only PDF	OCR explanation	Pass
Zero recognized resume skills	Score zero and missing JD labels	Pass
Duplicate visual points	Stable indices and correct marker	Pass

Table 7.2 Representative regression checks

7.8 What remains unmeasured

There is no ground-truth dataset of recruiter-approved equivalent skill pairs in this repository. Precision and recall for extraction, ranking quality, score calibration, end-to-end latency, and fairness across resume formats remain unmeasured. The current evidence is therefore a functional verification record, not a benchmark of hiring accuracy.

7.9 Limitations and responsible use

Technical limitations

The skill list determines which terms can enter the comparison. Long prose can contain terms in examples or disclaimers, and the section-focus rules cannot understand every document structure. The PDF reader requires selectable text and can misread columns or tables. A short label’s embedding can be unstable or misleading, and the 0.60 cutoff is a local choice. The three-dimensional projection loses information and may visually exaggerate or hide distance.

The server is a local prototype. It has no persistent analysis history, OCR, editable extraction review, user accounts, or deployment hardening. The output does not verify whether a listed skill was applied in work or at what level. A high score can coexist with poor evidence, missing requirements, or an irrelevant source document.

Interpretation rule

Use the number as a prompt to inspect the pair list and source documents. Do not use it alone to reject or select a person. When the recognized JD count seems low, fix the input or vocabulary before treating the score as informative. Human review remains necessary for context, experience, evidence, and role-specific requirements.

7.10 Privacy and evaluation boundaries

The web flow processes uploaded bytes in memory and does not save them as application data. The original notebooks can write vectors and score JSON into local output files, so a user running those notebooks should manage those artifacts according to their own privacy requirements. The project has no consented real-resume evaluation dataset and does not report personal outcomes.

A responsible next evaluation should use permissioned, de-identified documents; separate extraction labels from equivalence judgments; measure errors for rare and common skills; and compare the score with a clearly defined human review task. It should also examine whether the same term is merely mentioned, requested, or supported by experience. None of those checks is replaced by a visually appealing map.

Priority improvements

- Add an editable preview of recognized JD and resume skills before scoring.
- Broaden the alias vocabulary with real, reviewed examples and regression tests.
- Measure extraction and pair quality on consented, annotated documents before changing the score floor.

7.11 Reproducibility of the demonstration

The browser demonstration uses a synthetic JD with seven named requirements and a synthetic PDF listing five matching labels plus two additional technologies. It is repeatable because the input terms, current model, floor, and expected 5/7 score are documented. A later experimental study should store a fixed model revision and annotated evaluation corpus; the present report records only functional evidence from the local prototype.

CHAPTER 8
OUTPUT SCREENS

8.1 Two-screen user flow

The frontend has an upload screen and a result screen. The upload screen states the accepted formats and maximum sizes, shows the selected filenames, and submits both files to the local endpoint. The result screen opens after a successful response and presents the interactive map above the score and skill details. The same analysis object drives both the numeric result and the visual connections.

The interface uses lightweight entrance and score animations, ambient scene movement, and hover feedback. Users can pause rotation, reset the view, drag or use arrow keys to rotate, and scroll or use + and − to zoom. A reduced-motion preference pauses automatic movement. The score and skill rows remain in normal page content so the essential result is available without manipulating the map.

Figures 8.1 through 6.4 are screenshots of the running local application with synthetic demonstration files. They replace the previous document’s design mock-ups. The screenshots show actual upload, map, matched, and missing states; they are not evidence of performance on real hiring decisions.

8.2 Upload screen

The first screen uses two labeled drop zones for the JD and resume. The button becomes active after both files are present. The file format hints and size limits are visible before submission, and validation errors appear with the form. The screenshot below shows the synthetic demonstration files selected and ready for analysis.

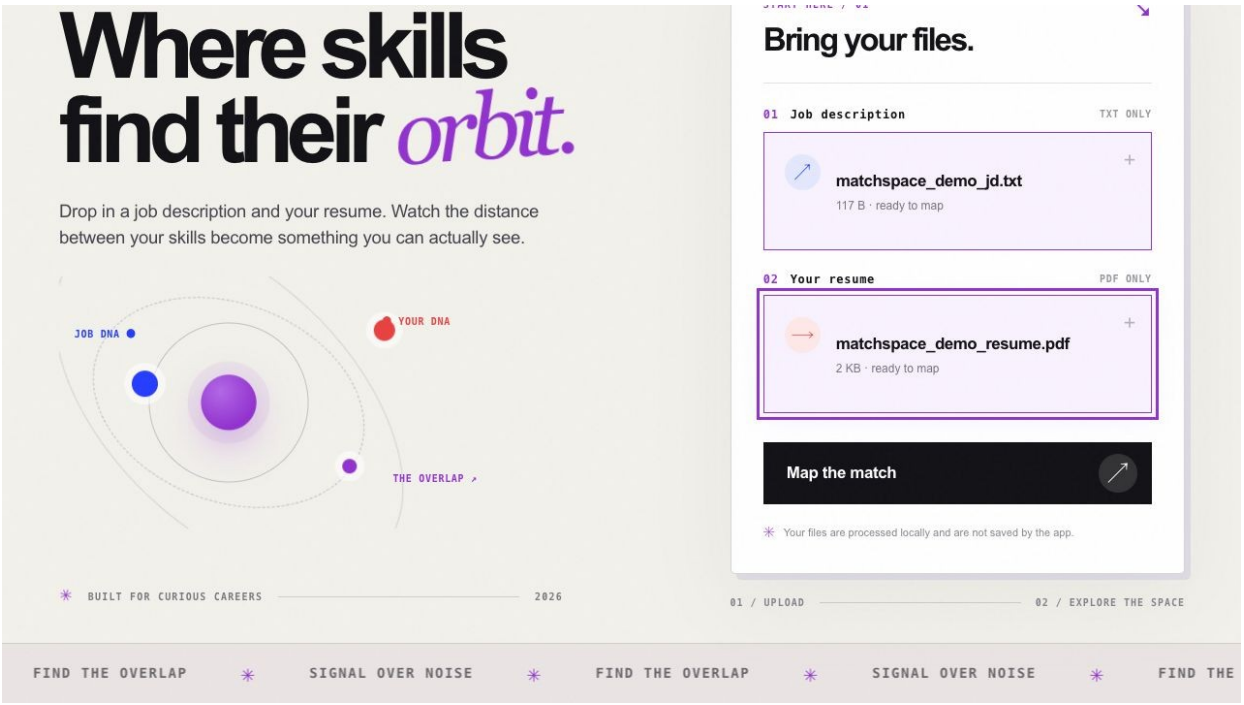


Figure 8.1 Upload screen with synthetic demonstration files selected

The files are passed to the local server for analysis. The web app does not persist the uploaded originals. A later analysis begins from a fresh selection.

8.3 Interactive skill map

The results page uses blue for JD points, red for resume points, and purple for selected scored pairs. The canvas supports rotation, zoom, point hover, and a pause control. Its legend states that the display is a three-dimensional projection and that scoring uses full vectors. This warning is essential because the projected spacing cannot prove equivalence by itself.

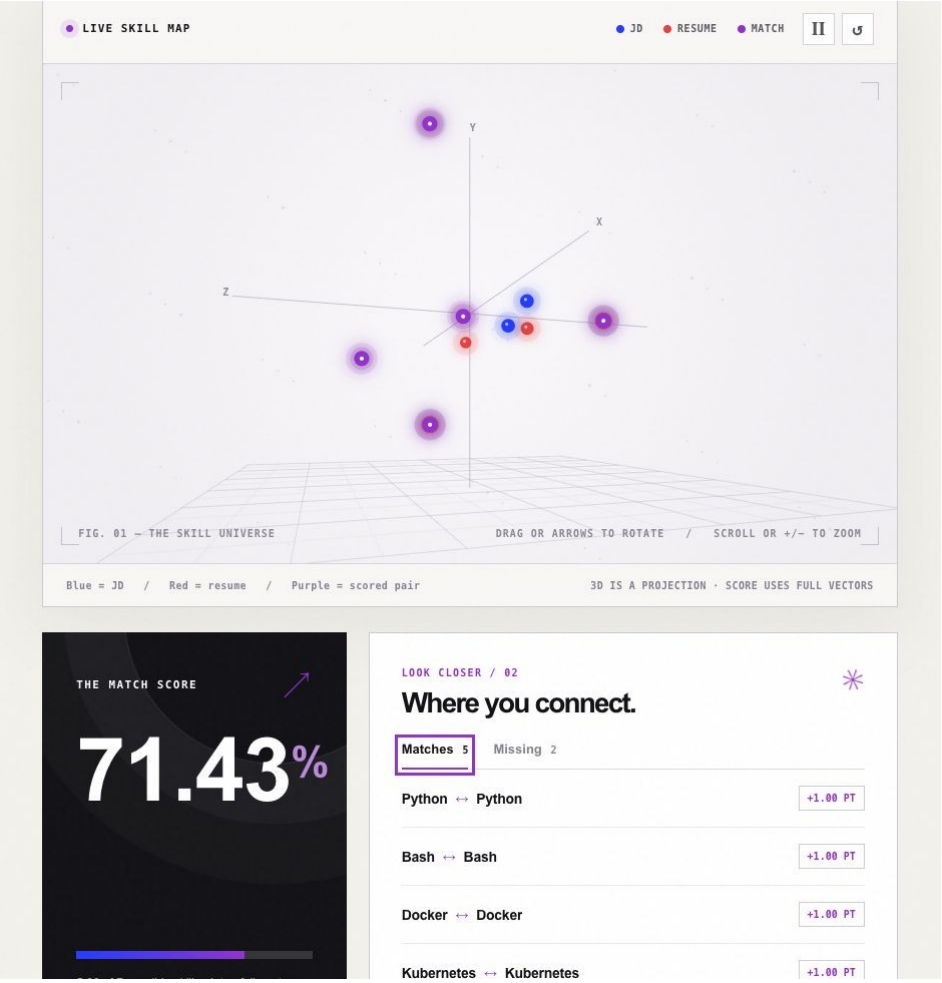


Figure 8.2 Running results screen showing the complete map, controls, and legend

The screen coordinates change as the user rotates the camera. The underlying pair assignment and score do not change with the camera angle.

8.4 Score and matched details

The score card reports the percentage, points earned, and denominator. The adjacent Matches tab lists each JD–resume pair and its awarded credit. In the synthetic example, five of seven recognized JD skills received one point, producing 71.43%. The count and rows make the denominator visible instead of asking the user to trust the percentage alone.

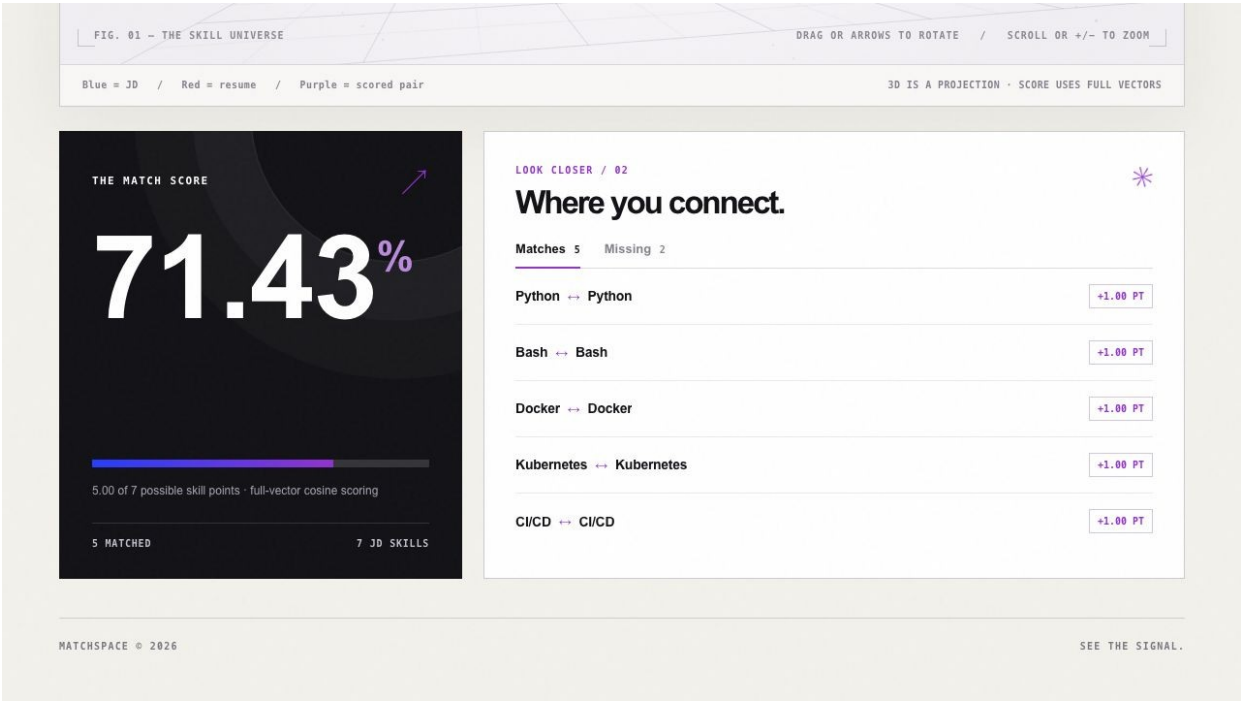


Figure 8.3 Score card and five matched skill pairs in the running frontend

8.5 Missing-skill details

The Missing tab lists recognized JD skills without a positive assigned pair. In the demonstration, Terraform and Observability remain unmatched. This means that the current extracted resume labels did not cross the configured scoring floor in the selected assignment; it is not a statement that the candidate cannot perform those tasks.

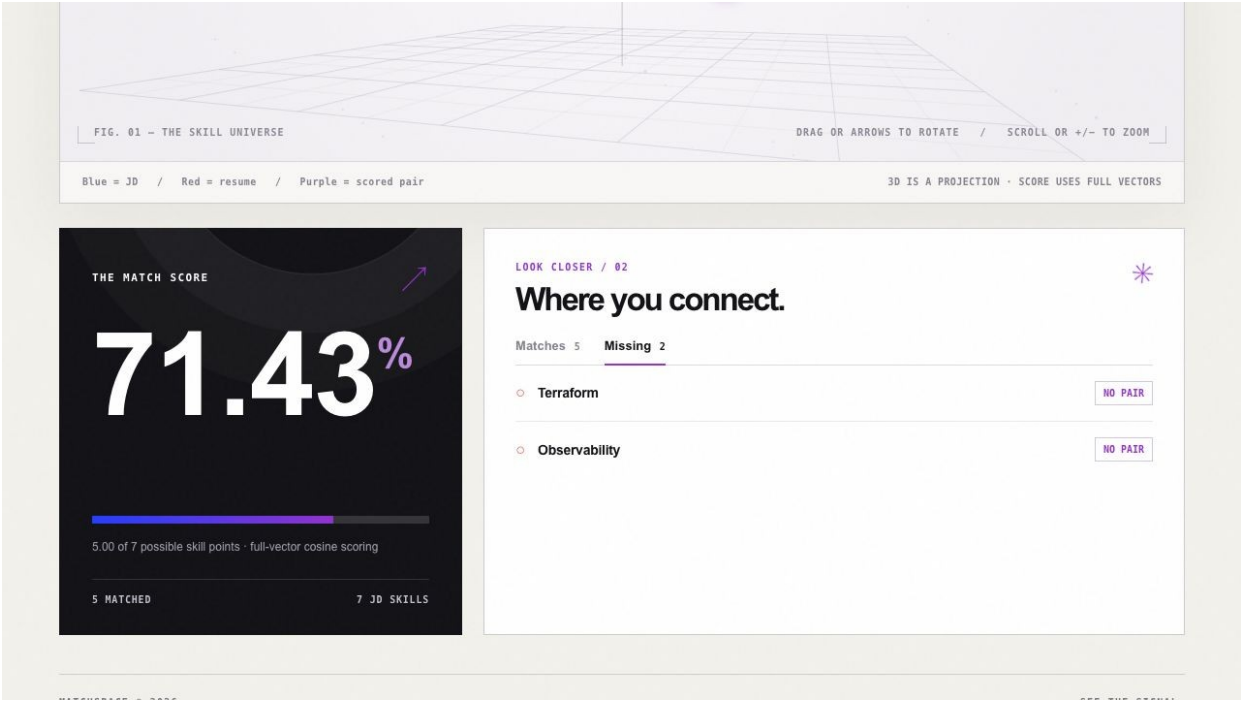


Figure 8.4 Missing tab for the same synthetic analysis

A user can return to the upload screen with New analysis. The browser does not keep a server-side history of previous reports.

CHAPTER 9
CONCLUSION

The implemented AI Resume Analyzer, branded Matchspace, accepts a text job description and selectable PDF resume, extracts named technical skills, embeds those skills with a pretrained model, and scores one-to-one pairs using full-vector cosine similarity. Its frontend provides the requested upload and result screens, with an interactive 3D projection, score card, and visible matched and missing labels.

The synthetic demonstration produced five points from seven recognized JD skills, or 71.43%. Thirty-one Python and six JavaScript tests passed for the tested behavior. The local prototype therefore completes its documented upload, comparison, and display flow for the verified cases. The result does not establish that the percentage is a hiring probability or that it generalizes to real applicant decisions.

The most useful next step is a reviewed extraction preview and a labeled evaluation set. Those would make the denominator easier to correct and provide evidence for any future threshold or model change. Until then, the tool should be used as an explainable comparison aid alongside human reading of the documents.

CHAPTER 10
FUTURE SCOPE

10.1 Reviewed extraction

The immediate product improvement is an editable preview of recognized skills before scoring. A user should be able to remove an example-only term, add a missed requirement, and see the final denominator before the score is computed. The review should preserve the original labels and show which aliases were applied, so corrections are traceable rather than hidden in a changed percentage.

10.2 Measured calibration

The next research step is a consented set of job descriptions and resumes with independently annotated skill mentions and equivalence judgments. That set can measure extraction precision and recall, one-to-one pairing quality, and the effect of different similarity floors. The threshold should be revised only against a defined review task, with held-out cases and error analysis by role and document format.

10.3 Document support

OCR and DOCX support would broaden the input range, but each additional parser needs checks for reading order, tables, columns, and missing text. A public deployment would also need explicit retention policy, security review, model and dependency pinning, and performance measurements. These are planned extensions, not completed parts of the current local prototype.

CHAPTER 11
REFERENCES

11.1 Project and research sources

[1] Sentence Transformers. all-MiniLM-L6-v2 model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[2] Sentence Transformers. Semantic Textual Similarity documentation. https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html

[3] SciPy. Linear sum assignment in Optimization tutorial. https://docs.scipy.org/doc/scipy/tutorial/optimize.html

[4] NumPy. numpy.linalg.svd documentation. https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html

[5] pypdf. Extract Text from a PDF. https://pypdf.readthedocs.io/en/stable/user/extract-text.html

[6] Python Software Foundation. http.server documentation. https://docs.python.org/3/library/http.server.html

[7] Mozilla Developer Network. FormData and Canvas API documentation. https://developer.mozilla.org/en-US/docs/Web/API/FormData and https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API

Project implementation and verification are documented in the repository README, matcher.py, server.py, frontend/, and tests/. External references explain the underlying tools; the measured project result comes from the local application and tests.

APPENDIX REPRODUCTION STEPS

A.1 Run the local application

```
uv venv my-venv
uv pip install --python my-venv/bin/python -r requirements.txt
my-venv/bin/python server.py
```

Open http://127.0.0.1:8000 and upload a UTF-8 .txt job description and selectable-text .pdf resume. The first accepted request may load or download the pretrained embedding model. The web flow does not write the uploaded originals to disk.

A.2 Verify the code

```
my-venv/bin/python -m unittest discover -s tests -v
node --test frontend/*.test.mjs
node --check frontend/app.js
```

The reported revision passed 31 Python tests and six JavaScript tests. Re-run these commands after changes to the vocabulary, embedding expansion, score floor, server contract, or frontend scene. Re-run notebook extraction before using any previously stored vector JSON after a matcher change.

A.3 Synthetic example

The JD used in Figures 8.1–6.4 lists Python, Bash, Docker, Kubernetes, Terraform, CI/CD, and Observability under Required Qualifications. The sample PDF lists Python, Bash, Docker, Kubernetes, CI/CD, Prometheus, and Linux. It was created solely to demonstrate the user flow and does not represent a real applicant.